NYCU-EE IC LAB – Spring2026

Lab01 Exercise

Design: Design Rule Check Accelerator (DRCA)

Data Preparation

1. Extract files from TA's directory:
   % tar -xvf ~iclabTA01/2026_S_LAB1.tar

Design Description and Examples

In the semiconductor industry, **Design Rule Checking (DRC)** is the essential verification process that ensures a physical layout complies with the specific manufacturing constraints of a foundry. These rules—covering parameters like minimum wire width, spacing, and enclosure—act as a bridge between the theoretical circuit design and the physical reality of lithography. DRC is critically important because it guarantees manufacturability and yield; even a sub-micron violation can lead to short circuits or broken traces, potentially ruining an entire wafer and resulting in massive financial losses. By implementing a DRC checker, you are building the "safety net" that translates complex geometric constraints into logical signals, ensuring that the final hardware is reliable and can be produced at scale.

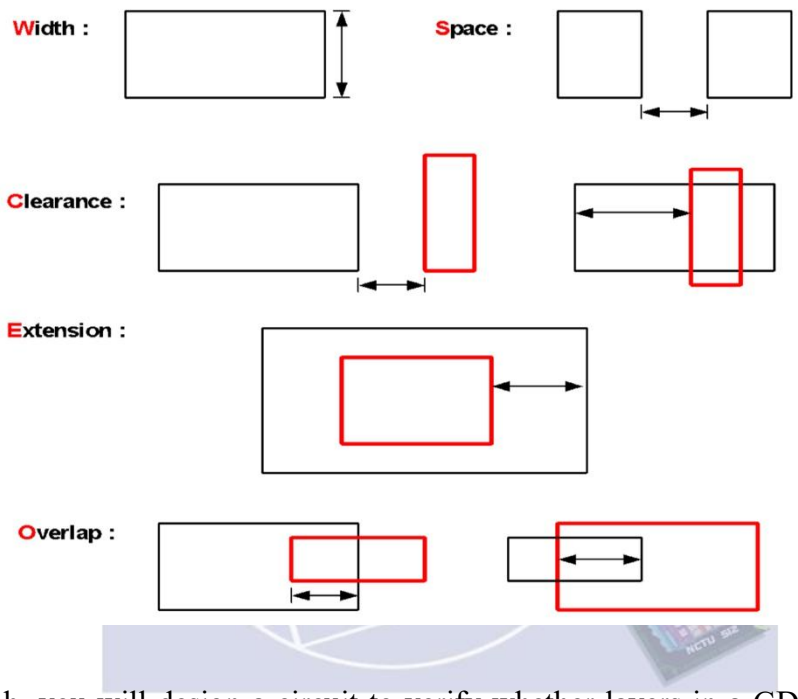
Before we start, let's learn about the definitions for some common terms used in the Design Rules



- **Width:** The minimum allowable dimension of a single shape on a specific layer. Ensuring proper width prevents wires from being too thin, which could lead to "opens" (breaks in the circuit) or overheating due to high resistance.
- **Space (Spacing):** The minimum required distance between two separate objects on the same layer. This rule is critical to prevent "shorts" (unintended electrical connections) between adjacent signals during the lithography process.
- **Clearance:** Similar to spacing, but often refers to the required distance between objects on *different* layers or between a signal and a specific boundary (like a keep-out zone). It ensures that features don't interfere with one another electrically or physically.

- **Extension:** The amount by which one layer must protrude beyond the edge of another layer it is covering or intersecting. For example, the Polysilicon gate must extend beyond the Diffusion area to ensure the transistor channel is fully formed.
- **Overlap:** The required amount of area where two different layers must cover each other. A common example is a "Via" or "Contact" that must be completely enclosed by a metal layer to ensure a solid electrical connection despite minor manufacturing misalignments.

To give you a sense, here are visual explanations for each term.



In this lab, you will design a circuit to verify whether layers in a GDS file violate a specific design rule. If violations detected, the circuit must report the total number of violations for each rule. To simplify the implementation, the following constraints have been established:

- **Coordinate Scaling:** The coordinates for every object are scaled to integers within a **16 × 16 grid**. All object coordinates will be integer values ranging from **0 to 15**.
- **Simplified Rules:** Design rule constraints are scaled to integer sizes. Note that these are provided for educational purposes and do not represent the actual design rules of a real-world fabrication process.
- Only 7 types of layers are considered in this lab.

1) CO (Contact): Connects OD or PO to M1.
2) OD (Diffusion): Active area.
3) PO (Poly): Polysilicon gate layer.
4) M1 (Metal 1): The first interconnect layer.
5) NP (N+): N-plus implant region.
6) PP (P+): P-plus implant region.
7) NW (N-Well): N-well region.

- **Geometric Constraints:** All shapes are assumed to be **rectangular**. They will be defined by the coordinates of their **lower-left** and **upper-right** corners.
- **Complexity Limit:** To bound the hardware requirements, no more than **8 shapes** of the same layer will be provided in one single input pattern.

Description for this lab

- **Input:**

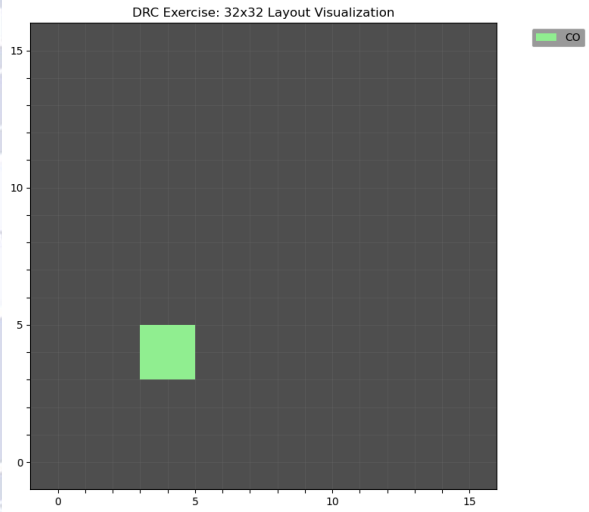
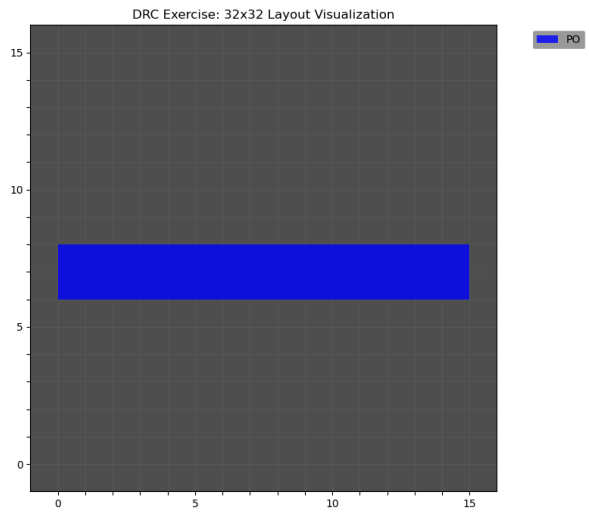
Your circuit will receive an **4-bit drc_sel signal** to determine the active design rule and **sixteen 19-bit shape signals** representing the physical layout.

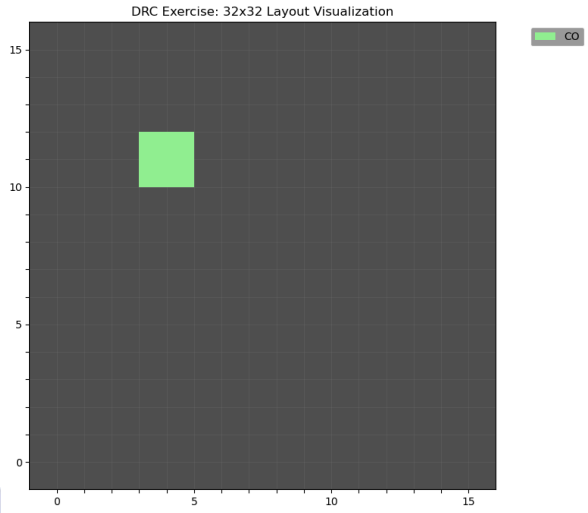
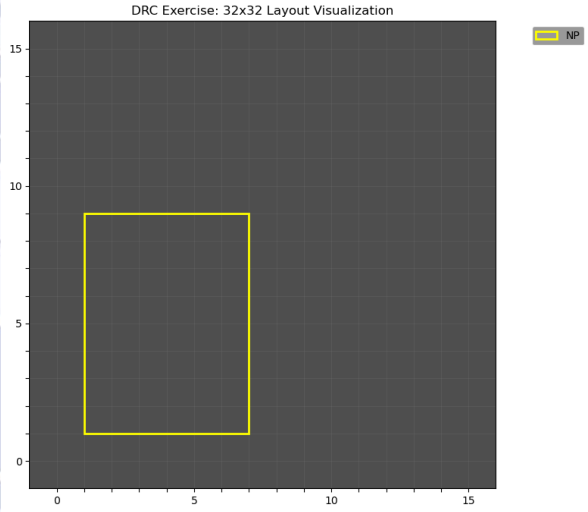
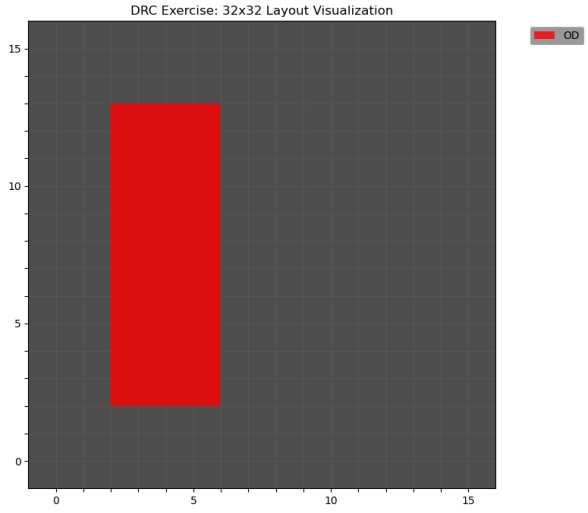
1. 4-Bit Design rule selection signal, ***drc_sel***

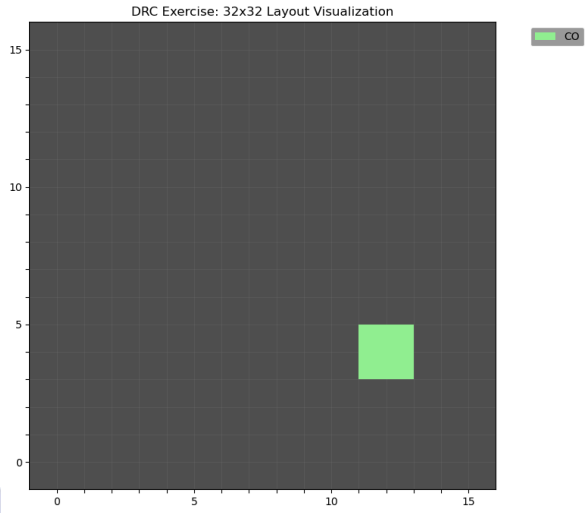
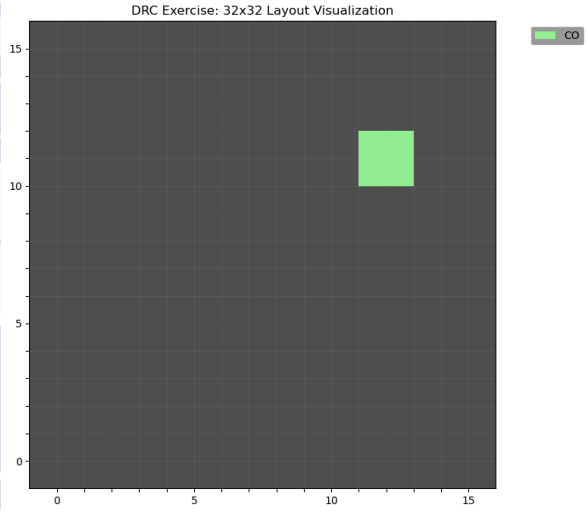
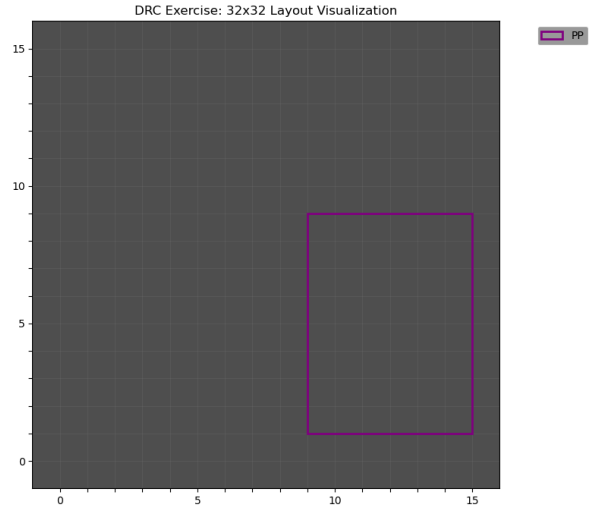
DRC_SEL[3:0]	Design rule name	Requirement
0000	Minimum contact width	>= 2
0001	Minimum contact spacing	>= 2
0010	Minimum diffusion width	>= 3
0011	Minimum diffusion spacing	>= 2
0100	Minimum Poly width	>= 2
0101	Minimum Poly spacing	>= 2
0110	Minimum M1 width	>= 3
0111	Minimum M1 spacing	>= 2
1000	Minimum N-plus implant width	>= 4
1001	Minimum N-plus implant spacing	>= 3
1010	Minimum P-plus implant width	>= 4
1011	Minimum P-plus implant spacing	>= 3
1100	Minimum N-well width	>= 5
1101	Minimum N-well spacing	>= 4

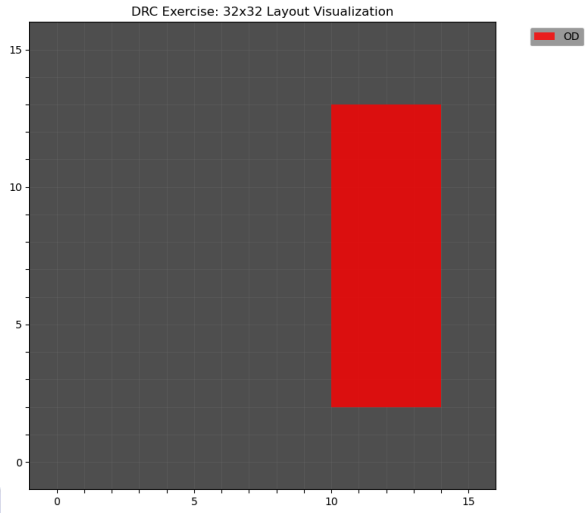
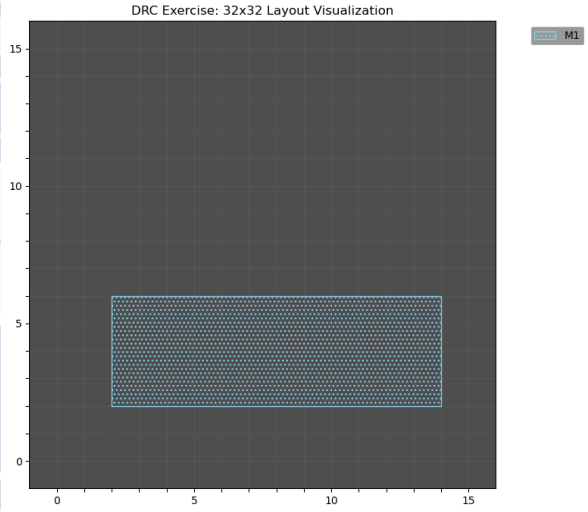
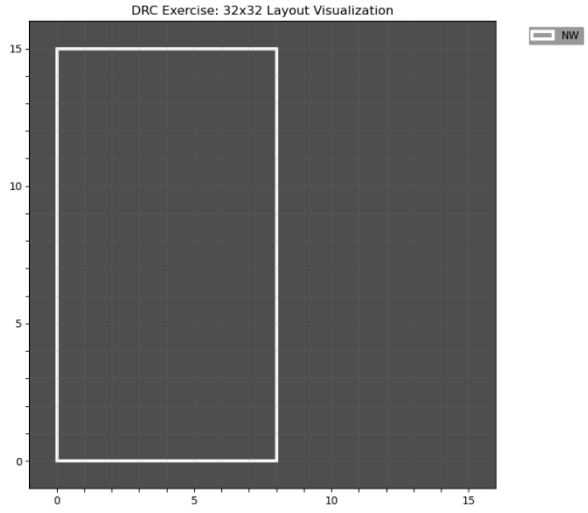
2. 19-Bit Shape Input Format, ***Shape0 ~ Shape15***.

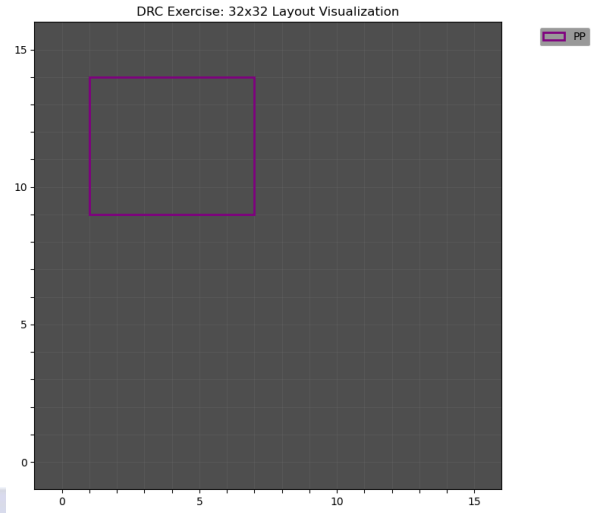
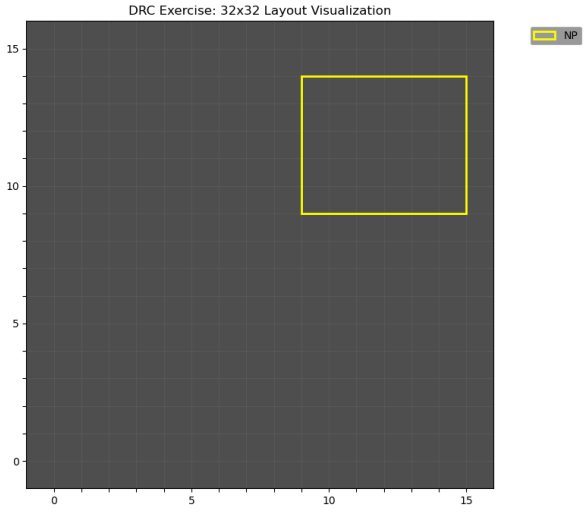
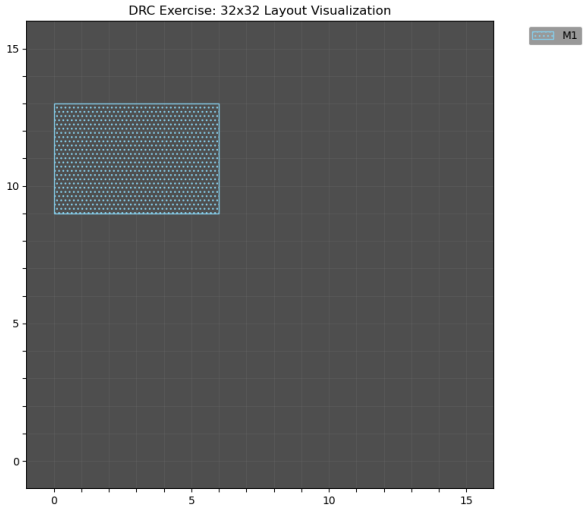
<table>
<tr><td>Layer type [18:16]

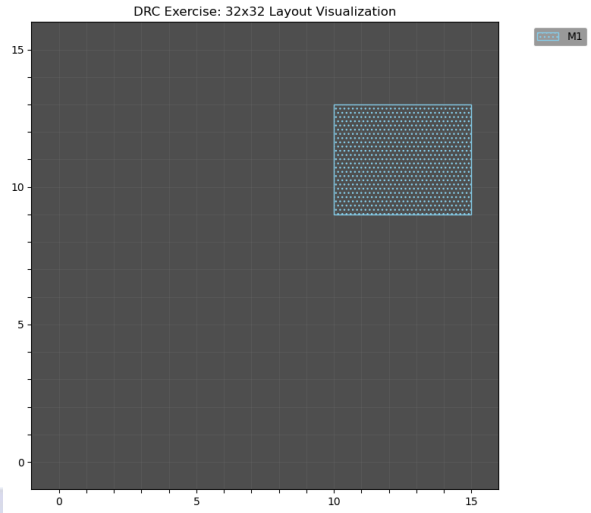
<table><tr><td>000</td><td>NONE</td></tr><tr><td>001</td><td>CO</td></tr><tr><td>010</td><td>OD</td></tr><tr><td>011</td><td>PO</td></tr><tr><td>100</td><td>M1</td></tr><tr><td>101</td><td>NP</td></tr><tr><td>110</td><td>PP</td></tr><tr><td>111</td><td>NW</td></tr></table></td><td>Lower left coordinate [15:8]
{x[3:0], y[3:0]}</td><td>Upper right coordinate [7:0]
{x[3:0], y[3:0]}</td></tr>
<tr><td>e.g. Shape1 3'd1</td><td>{4'd3, 4'd3}</td><td>{4'd5, 4'd5}</td></tr>
<tr><td>CO</td><td colspan="2"> 
</td></tr>
<tr><td>e.g. Shape2 3'd3</td><td>{4'd0, 4'd6}</td><td>{4'd15, 4'd8}</td></tr>
<tr><td>PO</td><td colspan="2"> 
</td></tr>
</table>

e.g. Shape3	3'd1	{4'd3, 4'd10}	{4'd5, 4'd12}
CO			
e.g. Shape4	3'd5	{4'd1, 4'd1}	{4'd7, 4'd9}
NP			
e.g. Shape5	3'd2	{4'd2, 4'd2}	{4'd6, 4'd13}
OD			

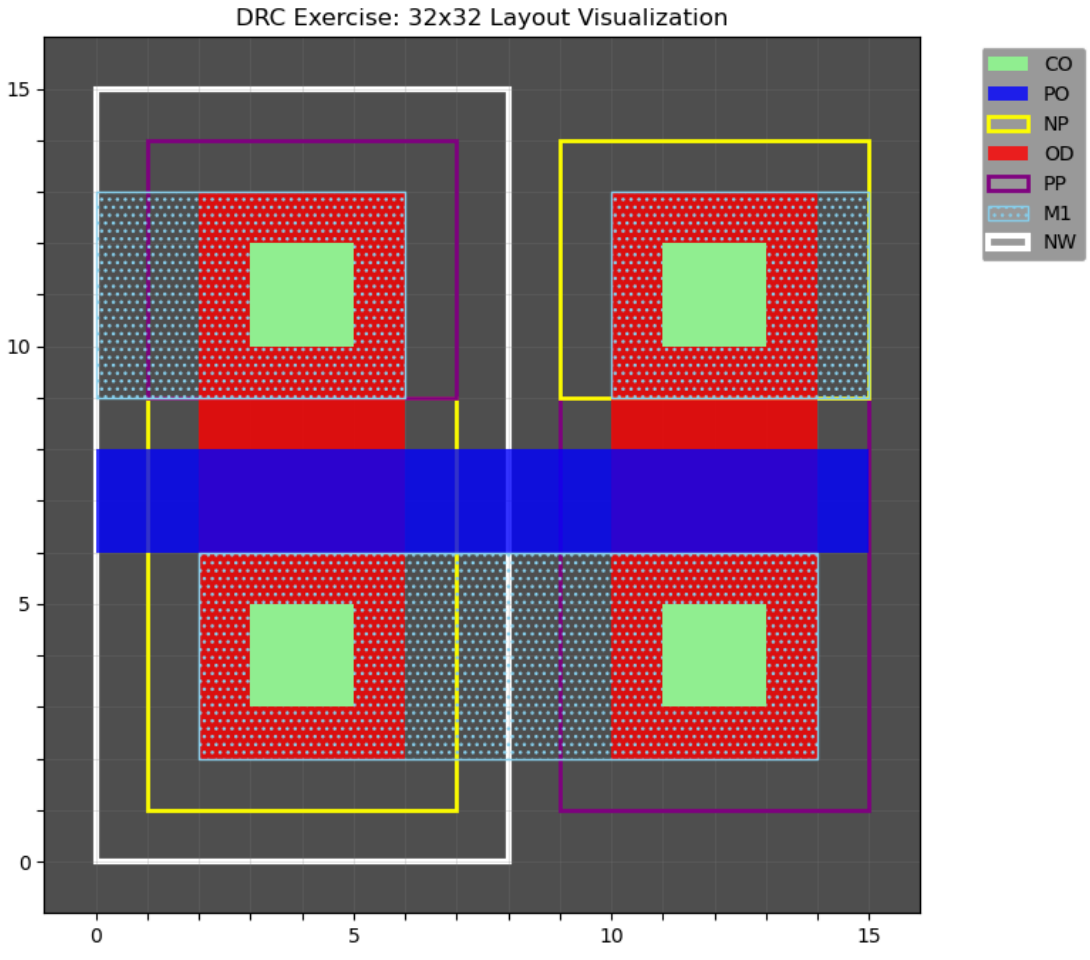
e.g. Shape6	3'd1	{4'd11, 4'd3}	{4'd13, 4'd5}
CO			
e.g. Shape7	3'd1	{4'd11, 4'd10}	{4'd13, 4'd12}
CO			
e.g. Shape8	3'd6	{4'd9, 4'd1}	{4'd15, 4'd9}
PP			

e.g. Shape9	3'd2	{4'd10, 4'd2}	{4'd14, 4'd13}
OD			
e.g. Shape10	3'd4	{4'd2, 4'd2}	{4'd14, 4'd6}
M1			
e.g. Shape11	3'd7	{4'd0, 4'd0}	{4'd8, 4'd15}
NW			

e.g. Shape12	3'd6	{4'd1, 4'd9}	{4'd7, 4'd14}
PP			
e.g. Shape13	3'd5	{4'd9, 4'd9}	{4'd15, 4'd14}
NP			
e.g. Shape14	3'd4	{4'd0, 4'd9}	{4'd6, 4'd13}
M1			

e.g. Shape15	3’d4	{4'd10, 4'd9}	{4'd15, 4'd13}
M1			
e.g. Shape16	3’d0	{4’d0, 4’d0}	{4’d0, 4’d0}
NONE			

Complete Layout (Inverter, without any DRC violation)



System Integration Silicon & Implementation

➢ Violation Calculate

You must calculate the total number of violations for the selected design rules and output the result as a 5-bit signal, **drc_result**. To ensure accurate reporting, please adhere to the following counting logic:

1. Width Violation Rule (Axis-Independent)

Width violations are evaluated independently for the X and Y dimensions. If a shape is undersized in both directions, it is treated as two distinct failure events.

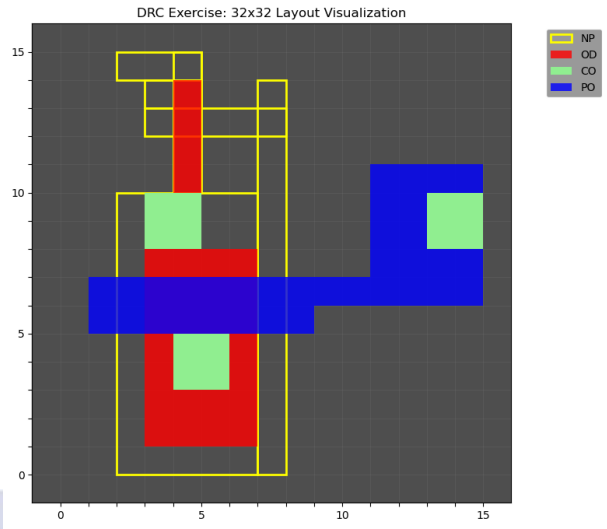
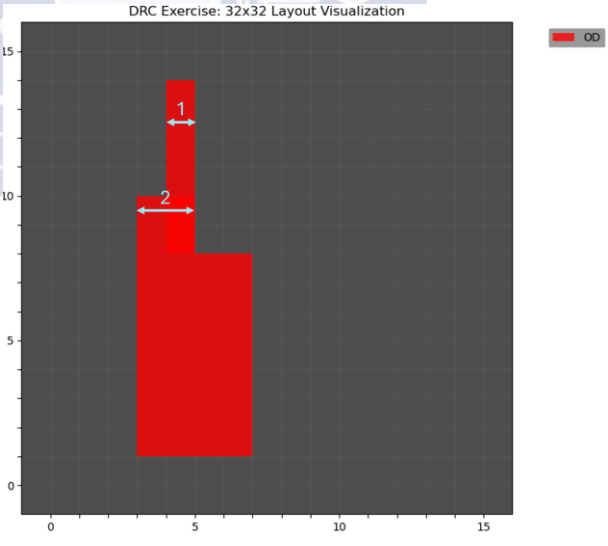
- **Logic**: Every instance where an object's dimension is less than the minimum requirement triggers an increment of the drc_result.
- **Dual-Axis Counting**: If a single shape violates the width requirement in both the X-axis and Y-axis (e.g., a 1x1 contact), the violation count increases by 2.
- **Overlap Handling**: Overlapping regions between shapes are treated as derived shapes. The resulting intersection area must independently satisfy the minimum width requirements. If the overlap region itself is "too thin" in either dimension, it triggers additional violations (check the following example for reference.)

2. Spacing Violation Rule (Rectilinear and Adjacent Only)

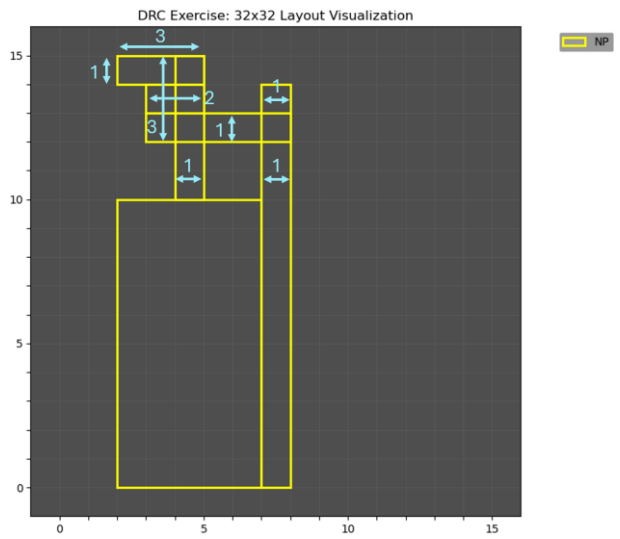
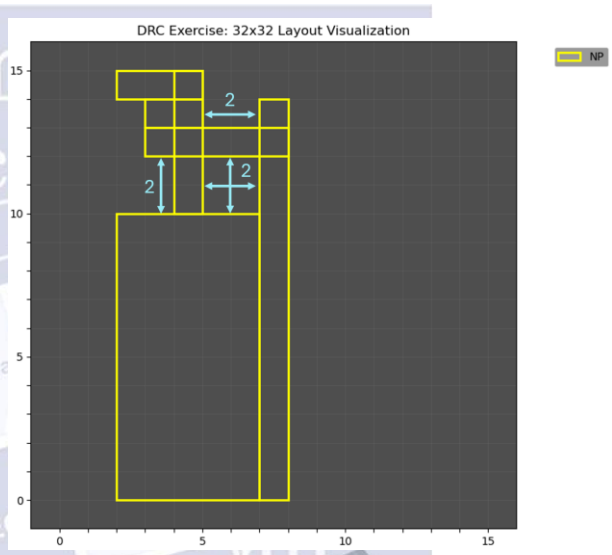
Spacing is defined as the distance between the parallel edges of two distinct objects. For the scope of this lab, **diagonal (corner-to-corner) spacing is not considered.**

- **Logic:** A violation is triggered if the distance, S, between two adjacent shapes is $0 < S < S_{min}$.
- **Dimensional Independence:** Spacing is checked strictly in the horizontal and vertical directions:
  - **Horizontal Spacing:** The air gap between the right edge of Shape A and the left edge of Shape B.
  - **Vertical Spacing:** The air gap between the top edge of Shape A and the bottom edge of Shape B.
- **Connectivity Agnostic:** The checker evaluates the distance between input array entries regardless of whether they are part of the same electrical net or logically merged.
- **Diagonal Exclusion:** Spacing is only checked where shapes have overlapping projections on the opposite axis (i.e., they are "facing" each other). Purely diagonal relationships that do not violate horizontal or vertical gaps are ignored.
- To prevent corner cases where a single spacing gap spans multiple shapes simultaneously, the following constraints apply to the **N-Well (NW) layer: the minimum required width is 2**. Any NW shape with a width less than 1 is considered non-physical and will be excluded from the test patterns.

<table>
<tr><td>Inputs

<table><tr><th>Shape#</th><th>Value (19’h)</th></tr><tr><td>1</td><td>5C3EF</td></tr><tr><td>2</td><td>53C8D</td></tr><tr><td>3</td><td>2485E</td></tr><tr><td>4</td><td>14365</td></tr><tr><td>5</td><td>52E5F</td></tr><tr><td>6</td><td>31596</td></tr><tr><td>7</td><td>1385A</td></tr><tr><td>8</td><td>5207A</td></tr><tr><td>9</td><td>2315A</td></tr><tr><td>10</td><td>316F7</td></tr><tr><td>11</td><td>5708E</td></tr><tr><td>12</td><td>1D8FA</td></tr><tr><td>13</td><td>54A5F</td></tr><tr><td>14</td><td>25178</td></tr></table></td><td colspan="2">Complete Layout
 
</td></tr>
<tr><td>drc_sel</td><td>Golden drc_result
(Expressed in decimal)</td><td>Visualization and description</td></tr>
<tr><td>0010</td><td>2</td><td>Assume OD appear under PO for simplicity
 
</td></tr>
</table>

System Integration & Silicon Implementation Group.

1000	8	
1001	4	

Inputs

Input Signal	Bit width	Description
drc_sel	4	4-bit inputs that determine the active design rule
shape1	19	Shape rectangle 1 [18:16] : Layer type [15:12] : Lower left corner x-coordinate [11:8] : Lower left corner y-coordinate [7:4] : Upper right corner x-coordinate [3:0] : Upper right corner y-coordinate
…		
shape16	19	Shape rectangle 15 [18:16] : Layer type

		[15:12] : Lower left corner x-coordinate [11:8] : Lower left corner y-coordinate [7:4] : Upper right corner x-coordinate [3:0] : Upper right corner y-coordinate

Outputs

Output Signal	Bit width	Description
drc_result	5	Represents the total drc violation for the selected design rule (Total violation won't exceed 31.)

Specifications

1. Top module name: DRCA (File name: DRCA.v)
2. The maximum total cell area should be smaller than 800,000

```
Number of ports:                          312
Number of nets:                         16233
Number of cells:                        14623
Number of combinational cells:          14623
Number of sequential cells:                 0
Number of macros/black boxes:               0
Number of buf/inv:                       2337
Number of references:                     134

Combinational area:             330328.860753
Buf/Inv area:                    23635.987132
Noncombinational area:               0.000000
Macro/Black Box area:                0.000000
Net Interconnect area:      undefined  (Wire load has zero net area)

Total cell area:                330328.860753
Total area:                 undefined
```

3. The clock cycle period for this lab should be smaller than 20ns.

```
1   `define CYCLE_TIME 10.0
2
3   module PATTERN (
4       output reg [3:0]  drc_sel,
5       output [18:0] shape0 ,
6       output [18:0] shape1 ,
```

00_TESTBED/Pattern.v

```
21  #============================
22  # (A) Global Parameters
23  #============================
24  set DESIGN "DRCA"
25  set CYCLE 10
26  #set INPUT_DLY 0
27  #set OUTPUT_DLY 0
```

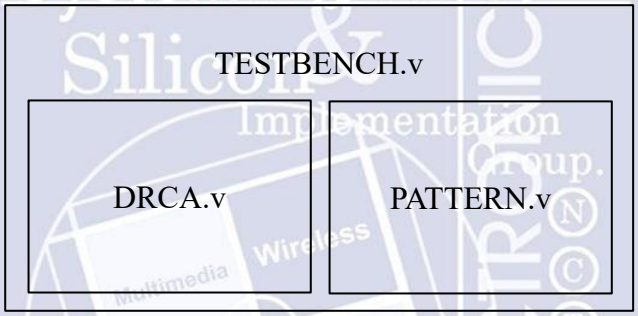
02_SYN/syn.tcl

4. After synthesis, check the "DRCA.area" and "DRCA.timing" in the folder

"Report". **The area report is valid only when the slack in the end of "DRCA.timing" is "MET".**

```
clock clk (rise edge)                          10.00       10.00
clock network delay (ideal)                     0.00       10.00
output external delay                           0.00       10.00
data required time                                         10.00
-----------------------------------------------------------------
data required time                                         10.00
data arrival time                                         -10.00
-----------------------------------------------------------------
slack (MET)                                                 0.00
```

Block Diagram



Grading Policy

The performance is determined by the area of your design. The less area your design has, the higher grade you get. Try to reach better performance by thinking your architecture before coding.

- Function Validity: 70%
- Performance: **Area * Cycle time** 30%

Students who successfully pass Lab01 during the first demonstration are eligible for 70% and may participate in the performance ranking to obtain the performance scores.
However, if the first demo is unsuccessful, a second attempt is permitted, but the total score for the entire lab will be capped at 50%, and the student will no longer be eligible for performance ranking.
Failure to pass both the first and second demonstrations will result in a score of zero for the lab. Finally, academic integrity is strictly enforced; any student found to have engaged in plagiarism within their code will automatically receive a score of zero.

Note

1. Tar all your design by run the command **Lab01/09_SUBMIT/00_tar**
2. Submit your design through **Lab01/09_SUBMIT/01_submit**
   a. 1st_demo deadline: 2026/03/09(Mon.) 12:00:00
   b. 2nd_demo deadline: 2026/03/11(Wed.) 12:00:00
3. If your file violates the naming rule, you will lose 5 points.
4. Don't use any wire/reg/submodule/parameter name called *error*, *Congratulations*, *latch* or *FAIL* otherwise you will fail the lab. Note: * means any char in front of or behind the word. e.g: error_note is forbidden.

Be careful about all details!

Template folders and reference commands:

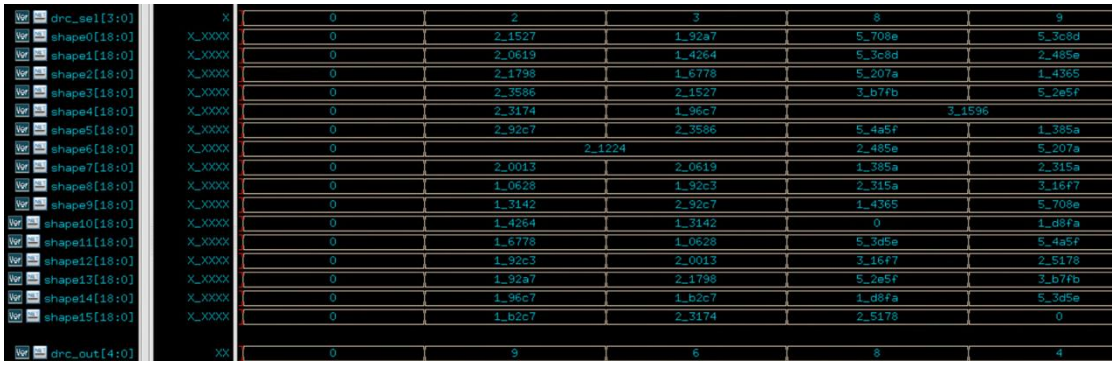
In demo, the reference commands is:

1. 01_RTL (RTL simulation):
   ./01_run_vcs_rtl
   ./print_layout
2. 02_SYN/ (Synthesis):
   ./01_run_dc_shell
   (Check the design's timing in /Report/ ADRC.timing)
   (Check the design's area in /Report/ ADRC.area)
3. 03_GATE/ (Gate-level simulation):
   ./01_run_vcs_gate
   ./print_layout
4. 09_SUBMIT/ (submit your files):
   ./00_tar
   ./01_submit
   ./02_check

You can key in **./09_clean_up** to clear all log files and dump files in each folder

Example Waveform

Input and output signal:



Hint

Hint1: Try to use **behavior modeling description** instead of gate level description.

Hint2: Try to use **submodule** rather than copy and paste to simply your design. (not necessary in this lab)

```
// ----------------------------------------------------

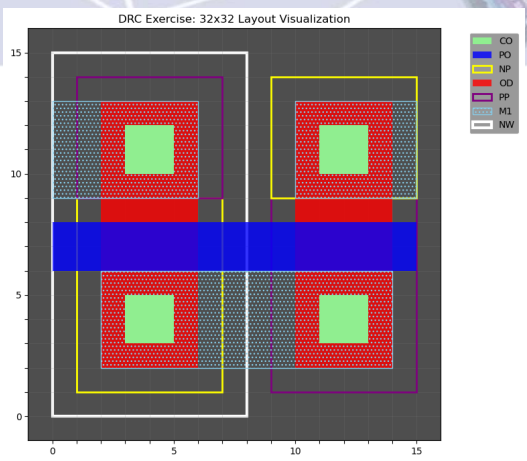
// Example for using submodule

// BBQ bbq0(.meat(meat_0), .vagetable(vagetable_0), .water(water_0),.cost(cost[0]));
```

Hint3: Try to think if there is any possible **hardware** that can be **shared** with different mode operation. You can use command dc_shell-gui to examine your design.(not necessary in this lab)

Hint4: TA provide a simple python code to visualize the testing layout. Copy and paste the input pattern for the 16 shapes to the 00_TESTBED/visual_layout.txt and enter ./print_layout to execute the python code.

```
1   1 3 3 5 5
2   3 0 6 15 8
3   1 3 10 5 12
4   5 1 1 7 9
5   2 2 2 6 13
6   1 11 3 13 5
7   1 11 10 13 12
8   6 9 1 15 9
9   2 10 2 14 13
10  4 2 2 14 6
11  7 0 0 8 15
12  6 1 9 7 14
13  5 9 9 15 14
14  4 0 9 6 13
15  4 10 9 15 13
16  0 0 0 0 0
```



Hint5: Pattern provided by TA will cover only some cases, you can try to write your own input by yourself. Here is the format how TA will read in PATTERN:

```
/* Input.txt format
2           Number of testing pattern

9           Golden_out
4           drc_sel
2 1 5 2 7   shape1 layer_type lower-left corner (x y) upper-right corner (x y) All in decimal, seperated by one sigle space.
3 0 6 1 9   shape2 layer_type lower-left corner (x y) upper-right corner (x y) All in decimal, seperated by one sigle space.
7 1 7 9 8   shape3 layer_type lower-left corner (x y) upper-right corner (x y) All in decimal, seperated by one sigle space.
0 0 0 0 0
...
5 9 2 11 15 shape15 layer_type lower-left corner (x y) upper-right corner (x y) All in decimal, seperated by one sigle space.

7           Golden_out (Next pattern)
...
*/
```

You can check input.txt and PATTERN.v in 00_TESTBED as a reference, and choose to write either c++/python or Verilog code for generating testing pattern.

